

Practical 2

Aim: Implementation and Time analysis of linear and binary search algorithms.

Algorithms:

Practical-2
linear search

Algorithms

Step 1: start

Step 2: Set $i=0$

Step 3: Repeat while $i < n$
if $A[i] == key$
Return i
end if
 $i = i + 1$
end while

Step 4: Return -1

Step 5: stop

Binary search

Algorithm binary search (A, n, key)

Step 1: start

Step 2: set $low = 0$
set $high = n$

Step 3: Repeat while $low <= high$
 $mid = (low + high) / 2$
if $A[mid] == key$
 $low = mid + 1$
else if $A[mid] < key$
 $low = mid + 1$
else
 $high = mid - 1$
endif
steps: stop

Set P4: Return -1

Code:

```
#include <iostream> using
namespace std;
// Linear Search
int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key)
            return i;
    }
    return -1;
}

// Binary Search
int binarySearch(int arr[], int n, int key)
{
    int left = 0;
    int right = n - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            left = mid +
1;
        else
            right = mid - 1;
    }

    return -1;
}

int main() {
int arr[] = {10, 20, 30, 40, 50, 60, 70};
int n = sizeof(arr) / sizeof(arr[0]);
```

```
    int key;
    cout << "Enter element to search: ";
    cin >> key;

    // Linear Search
    int linearResult = linearSearch(arr, n, key);

    if (linearResult != -1)
        cout << "Linear Search: Element found at index "
            << linearResult << endl;
    else
        cout << "Linear Search: Element not found << endl;

    // Binary Search
    int binaryResult = binarySearch(arr, n, key);

    if (binaryResult != -1)
        cout << "Binary Search: Element found at index "
            << binaryResult << endl;
    else
        cout << "Binary Search: Element not found" << endl;

    return 0;
}
```

Output:

```
Enter element to search: 60
Linear Search: Element found at index 5
Binary Search: Element found at index 5

=== Code Execution Successful ===
```

Time Complexities:

Time complexities: Linear Search

```

int (i=0, i<arr[i]) {
    if (arr[i]==key)
        return i;
}
T(n) = n + c
Ignore constant terms
T(n) = O(n)

```

Best case:
If key found at first position
 $T(n) = 1$
 $T(n) = O(1)$

Worst case:
If key is at last position
 $T(n) = n$
 $T(n) = O(n)$

Average case:
on average it checks about half the array
 $T(n) = \frac{n}{2}$
 $T(n) = O(n)$

Time complexities: Binary Search

1. best case
 $T(n) = O(1)$

2. Avg case
 $T(n) = O(\log n)$

3. worst case
 $T(n) = O(\log n)$

$T(n) = \begin{cases} 1 & n=1 \\ T(\frac{n}{2}) + 1 & n>1 \end{cases}$
 $T(n) = T(\frac{n}{2}) + 1 - \text{①}$
 $n \Rightarrow \frac{n}{2}$
 $T(\frac{n}{2}) = T(\frac{n}{4}) + 1 - \text{②}$
 ① in ②
 $T(n) = T(\frac{n}{2^2}) + 2 - \text{③}$
 $n \Rightarrow \frac{n}{4}$
 $T(\frac{n}{4}) = T(\frac{n}{8}) + 1 - \text{④}$
 ③ in ④
 $T(n) = T(\frac{n}{2^3}) + 3$
 $T(n) = T(\frac{n}{2^k}) + k$
 $n \Rightarrow \frac{n}{2^k}$
 $n = \frac{n}{2^k}$
 $n = 2^k$
 $k = \log_2 n$
 $T(n) = T(\frac{n}{n}) + \log_2 n$
 $T(n) = T(1) + \log_2 n$
 $T(n) = 1 + \log_2 n$
 $T(n) = O(\log n)$